

10. Automating Application Deployment Using a CI/CD Pipeline

Go to <https://killercode.com/docker/scenario/container-build> and perform the following tasks:

Simulate a Basic CI/CD Pipeline Locally (In PWD)

Let's automate these steps:

1. Build a Docker image
 2. Run tests (if any)
 3. Push to a local "registry" or deploy container
 4. Restart the container
-

1. Sample Project Structure

```
myapp/  
├── app.py  
├── Dockerfile  
├── requirements.txt  
└── deploy.sh
```

Dockerfile (vi Dockerfile)

```
FROM python:3.10-slim  
WORKDIR /app  
COPY . .  
RUN pip install -r requirements.txt  
CMD ["python3", "app.py"]
```

 app.py (vi app.py)

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from CI/CD automated container!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

 requirements.txt (vi requirements.txt)

```
flask
```

 deploy.sh (CI/CD automation) (vi deploy.sh)

```
#!/bin/bash

PORT=8080

echo "Building Docker image..."
docker build -t myapp:latest .

echo "Stopping old containers..."
docker stop myapp || true
```

```
docker rm myapp || true
```

```
echo "Running new container on port $PORT..."
```

```
docker run -d --name myapp -p $PORT:8080 myapp:latest
```

```
echo "App deployed!"
```

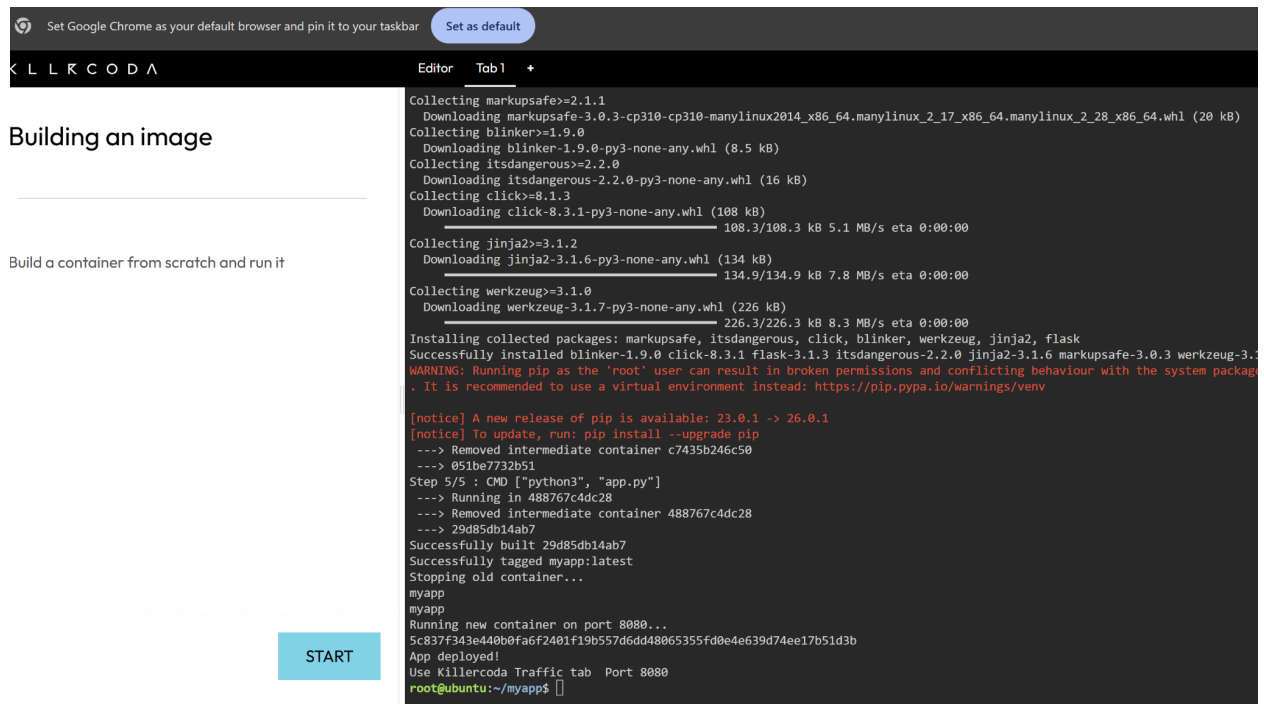
```
echo "Use Killercode Traffic tab Port $PORT"
```

Make it executable:

```
chmod +x deploy.sh
```

Run:

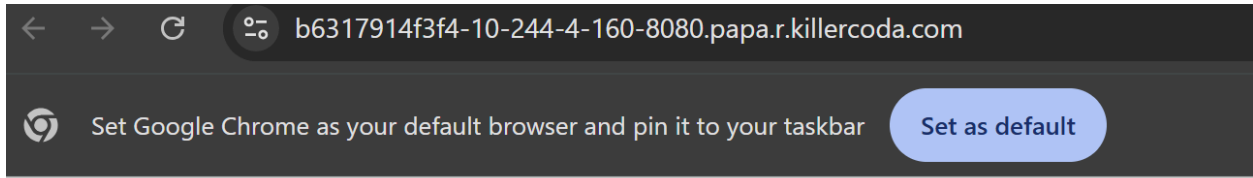
```
./deploy.sh
```



The screenshot shows the Killercode IDE interface. On the left, there's a sidebar with the text "Building an image" and "Build a container from scratch and run it". The main area is a terminal window displaying the output of the `./deploy.sh` script. The terminal output shows the installation of various Python packages (markupsafe, blinker, itsdangerous, click, werkzeug, jinja2, flask) and the successful building and running of a Docker container named `myapp` on port 8080. The terminal prompt is `root@ubuntu:~/myapp$`.

```
Collecting markupsafe>=2.1.1
  Downloading markupsafe-3.0.3-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)
Collecting blinker>=1.9.0
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting itsdangerous>=2.2.0
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting click>=8.1.3
  Downloading click-8.3.1-py3-none-any.whl (108 kB)
Collecting jinja2>=3.1.2
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Collecting werkzeug>=3.1.0
  Downloading werkzeug-3.1.7-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.3.1 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.7
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv

[notice] A new release of pip is available: 23.0.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
---> Removed intermediate container c7435b246c50
---> 051be7732b51
Step 5/5 : CMD ["python3", "app.py"]
---> Running in 488767c4dc28
---> Removed intermediate container 488767c4dc28
---> 29d85db14ab7
Successfully built 29d85db14ab7
Successfully tagged myapp:latest
Stopping old container...
myapp
myapp
Running new container on port 8080...
5c837f343e440b0fa6f2401f19b557d6dd48065355fd0e4e639d74ee17b51d3b
App deployed!
Use Killercode Traffic tab Port 8080
root@ubuntu:~/myapp$
```



Hello from CI/CD automated container!

//older version' s deploy.sh

```
#!/bin/bash
```

```
echo " Building Docker image..."
```

```
docker build -t myapp:latest .
```

```
echo " Stopping old container (if running)..."
```

```
docker stop myapp || true
```

```
docker rm myapp || true
```

```
echo " Running new container..."
```

```
docker run -d --name myapp -p 5000:5000 myapp:latest
```

```
echo " App deployed at:"
```

```
echo "http://$(hostname -i):5000"
```

👁️ Access the App

If you're in labs.play-with-docker:

- You'll see a **5000** port badge (click it)
- Or open manually:

`http://ip-XXXX-XX-XX-XX.play-with-docker.com:5000`

✅ Step-by-Step Troubleshooting

- ♦ 1. List all containers, even stopped ones

`docker ps -a`

You'll likely see your container there with a **STATUS** like **Exited (1)** or similar.

- ♦ 2. Check logs for why it crashed

`docker logs <container_id>`

This will show exactly why the container failed — could be:

- Missing file
- Syntax error in Python
- Python not found

- Flask crash

-
- ♦ 3. Restart the container manually (optional)

```
docker start <container_id>
```

- ♦ 4. Or rebuild and re-run it cleanly

```
docker build -t flask-app .  
docker run -d -p 5000:5000 flask-app
```

OUTPUT:

```
#####  
#                               WARNING!!!!                               #  
# This is a sandbox environment. Using personal credentials             #  
# is HIGHLY! discouraged. Any consequences of doing so are             #  
# completely the user's responsibilities.                                #  
#                               #                                         #  
# The PWD team.                                                         #  
#####  
[node1] (local) root@192.168.0.18 ~  
$ mkdir myapp/  
[node1] (local) root@192.168.0.18 ~  
$ cd myapp/  
[node1] (local) root@192.168.0.18 ~/myapp  
$ vi app.py
```

```
[node1] (local) root@192.168.0.18 ~/myapp  
$ vi Dockerfile  
[node1] (local) root@192.168.0.18 ~/myapp
```

```
[node1] (local) root@192.168.0.18 ~/myapp  
$ vi requirements.txt
```

```
[node1] (local) root@192.168.0.18 ~/myapp  
$ vi deploy.sh
```

```

$ chmod +x deploy.sh
[node1] (local) root@192.168.0.18 ~/myapp
$ ./deploy.sh
[+] Building Docker image...
[+] Building 10.5s (9/9) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 144B                             0.0s
=> [internal] load metadata for docker.io/library/python:3.10-slim 1.5s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 813B                                  0.0s
=> [1/4] FROM docker.io/library/python:3.10-slim@sha256:65c843653048a3ba22c8d5083a022f44aef774974f0f7f70c 4.8s
=> => resolve docker.io/library/python:3.10-slim@sha256:65c843653048a3ba22c8d5083a022f44aef774974f0f7f70c 0.0s
=> => sha256:d7c79db7d957a3cfbc9d3930da23f24b9d988753b0bcbee3185cb7a76657fee3 5.31kB / 5.31kB 0.0s
=> => sha256:8a628cdd7ccc83e90e5a95888fcb0ec24b991141176c515ad101f12d6433eb96 28.23MB / 28.23MB 0.6s
=> => sha256:c96cb1c893456d3b64ac93cbebbba12262429b64eca53584506f09ae223d1efe8 3.51MB / 3.51MB 0.4s

```

```

=> => extracting sha256:29e6ce9bbfedce1cc78818c9d0632f7e6f06c35278be1c3b0d979cd2b6b6c26b 0.0s
=> [2/4] WORKDIR /app                                           0.0s
=> [3/4] COPY . .                                               0.0s
=> [4/4] RUN pip install -r requirements.txt                    3.8s
=> exporting to image                                           0.2s
=> => exporting layers                                           0.2s
=> => writing image sha256:72fb7666b5c448f05f717225c8981333907aceb05e3e8a9994639aa5ebe6dfed 0.0s
=> => naming to docker.io/library/myapp:latest                 0.0s
[+] Stopping old container (if running)...
Error response from daemon: No such container: myapp
Error response from daemon: No such container: myapp
[+] Running new container...
9c92a723bfa62213cd041af8b089ef0f799b771e2bc429fded2607498a95c4eb
[+] App deployed at:
http://192.168.0.18:5000
[node1] (local) root@192.168.0.18 ~/myapp
$

```

< > ↻ Not secure ip172-18-0-10-d02u1ciim2rg0081sg00-5000.direct.labs.play-with-docker.com 🔍 ⬇️ 👤 Guest

Hello from CI/CD automated container!